TECHNICAL DESIGN DOCUMENT

GraphNotes

Phases 1–3 · MVP Core: Vault Engine, Editor & Rendering, Linking Engine

Document Version	1.0
Status	Draft
Date	June 2026
Scope	Phases 1–3 (MVP Core)
Stack	Tauri 2.0 · Rust · React/TypeScript · SurrealDB
Author	GraphNotes Engineering

CONFIDENTIAL | Version 1.0 | June 2026

Table of Contents

1. Introduction

1.1 Purpose

This Technical Design Document (TDD) specifies the exact implementation blueprint for Phases 1 through 3 of GraphNotes — the MVP core encompassing the Vault Engine, the Editor and Rendering stack, and the Linking Engine. It is a companion document to the Architecture Design Document (ADD v2.0) and the Software Requirements Specification (SRS v2.0), translating their architectural decisions and functional requirements into concrete Rust structs, function signatures, data flows, IPC contracts, and edge-case handling rules.

Engineers should read this document before writing any code for the covered phases. The ADD answers "what is the system"; this TDD answers "how exactly do we build it".

1.2 Scope

This document covers the following implementation phases:

- Phase 1 — Core Vault Engine: SurrealDB initialization, vault open/scan, frontmatter parsing, note CRUD, and the note-list UI.
- Phase 2 — Editor and Rendering: CodeMirror 6 integration, Markdown preview, KaTeX math rendering, and the theme system.
- Phase 3 — Linking Engine: Wikilink parsing, graph edge creation, backlinks panel, dangling links, cascade rename, and transclusion.

Phases 4–10 (Graph Visualization, Search, Templates, LLM, GraphRAG, Android, Polish) are out of scope for this version of the TDD.

1.3 Relationship to Other Documents

Document	Role
SRS v2.0	Defines what the system must do (functional and non-functional requirements)
Architecture Design Document v2.0	Defines the layered system structure, technology stack, and IPC patterns
Implementation Checklist v2.0	Provides the ordered task list derived from this TDD
This TDD (Phases 1–3)	Specifies exact structs, signatures, data flows, and edge cases for implementation

1.4 Conventions

- Rust function signatures are shown with full async fn signatures and return types.
- TypeScript IPC types mirror the Rust structs and are maintained in src/types/ipc.ts.
- SurrealQL queries are shown verbatim as they appear in the codebase.
- Edge cases are highlighted in ⚠ **Edge Case** callout boxes.
- All IPC command names follow the domain_action convention defined in the ADD.

2. Shared Infrastructure

2.1 AppState

AppState is the central shared-state struct held by Tauri's state manager. It is initialized once in main.rs and injected into every Tauri command handler via tauri::State<'_, AppState>.

2.1.1 Struct Definition (src-tauri/src/state.rs)

```
use std::sync::Arc;
use std::path::PathBuf;
use tokio::sync::RwLock;
use surrealdb::Surreal;
use surrealdb::engine::local::SurrealKv;
use notify::RecommendedWatcher;

pub struct AppState {
    /// Shared SurrealDB handle — internally Arc, no external lock needed
    pub db: Arc<Surreal<SurrealKv>>,

    /// Currently open vault root path (None if no vault open)
    pub vault_path: Arc<RwLock<Option<PathBuf>>>,

    /// notify file watcher handle — dropped to stop watching
    pub watcher: Arc<RwLock<Option<RecommendedWatcher>>>,
}
```

Design Note: *Surreal<SurrealKv> manages its own internal locking; wrapping in Arc is sufficient. Mutable fields use tokio::sync::RwLock rather than std::sync::Mutex to remain async-compatible.*

2.1.2 Initialization (src-tauri/src/main.rs)

```
#[tokio::main]
async fn main() {
    tauri::Builder::default()
        .setup(|app| {
            let app_data = app.path().app_data_dir()?;
            std::fs::create_dir_all(&app_data)?;

            let db = tauri::async_runtime::block_on(async {
                let db =
Surreal::new::<SurrealKv>(app_data.join("graphnotes.db")).await?;
                db.use_ns("graphnotes").use_db("vault").await?;
                db::migrations::run(&db).await?;
                Ok::<_, surrealdb::Error>(Arc::new(db))
            })?;

            app.manage(AppState {
                db,
                vault_path: Arc::new(RwLock::new(None)),
                watcher: Arc::new(RwLock::new(None)),
            });
            Ok(())
        })
        .invoke_handler(tauri::generate_handler![
            commands::vault::vault_open,
            commands::vault::vault_close,
            commands::vault::vault_rebuild,
            commands::notes::note_list,
```

```
            commands::notes::note_read,
            commands::notes::note_save,
            commands::notes::note_create,
            commands::notes::note_delete,
            commands::notes::note_rename,
            commands::graph::graph_query_backlinks,
            commands::graph::graph_query_forward_links,
            commands::graph::graph_query_unlinked_mentions,
        ])
        .run(tauri::generate_context!())
        .expect("error running GraphNotes");
}
```

2.2 IPC Payload Types

All IPC payloads are Rust structs with #[derive(Serialize, Deserialize, Debug, Clone)]. Matching TypeScript definitions are maintained in src/types/ipc.ts. The canonical source of truth is the Rust side.

2.2.1 Core IPC Types (src-tauri/src/types.rs)

```
use serde::{Serialize, Deserialize};
use std::collections::HashMap;

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct VaultInfo {
    pub path: String,
    pub note_count: u64,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct NoteSummary {
    pub id: String,
    pub path: String,
    pub title: String,
    pub updated_at: String,   // ISO 8601
    pub tags: Vec<String>,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct NoteRecord {
    pub id: String,
    pub path: String,
    pub title: String,
    pub content: String,
    pub frontmatter: HashMap<String, serde_json::Value>,
    pub created_at: String,
    pub updated_at: String,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct BacklinkEntry {
    pub source_path: String,
    pub source_title: String,
    pub snippet: String,      // 1-2 lines surrounding the wikilink
}

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct ProgressPayload {
    pub pct: u8,              // 0-100
    pub message: String,
```

```
}

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct RefactorResult {
    pub files_updated: u32,
}
```

2.2.2 TypeScript Mirror (src/types/ipc.ts)

```
export interface VaultInfo { path: string; note_count: number; }

export interface NoteSummary {
  id: string; path: string; title: string;
  updated_at: string; tags: string[];
}

export interface NoteRecord {
  id: string; path: string; title: string; content: string;
  frontmatter: Record<string, unknown>;
  created_at: string; updated_at: string;
}

export interface BacklinkEntry {
  source_path: string; source_title: string; snippet: string;
}

export interface ProgressPayload { pct: number; message: string; }
export interface RefactorResult  { files_updated: number; }
```

2.3 SurrealDB Schema Migrations

Schema DDL runs once on startup via a migrations module. All DEFINE statements are idempotent — safe to re-run against an existing database.

2.3.1 Migration Runner (src-tauri/src/db/mod.rs)

```
pub async fn run(db: &Surreal<SurrealKv>) -> Result<(), surrealdb::Error> {
    db.query(include_str!("schema.surql")).await?;
    Ok(())
}
```

2.3.2 Schema DDL (src-tauri/src/db/schema.surql)

```
-- ── Notes ──────────────────────────────────────────────────────
DEFINE TABLE note SCHEMAFULL;
DEFINE FIELD path         ON note TYPE string;
DEFINE FIELD title        ON note TYPE string;
DEFINE FIELD content      ON note TYPE string;
DEFINE FIELD frontmatter  ON note TYPE object;
DEFINE FIELD created_at   ON note TYPE datetime;
DEFINE FIELD updated_at   ON note TYPE datetime;
DEFINE INDEX note_path    ON note COLUMNS path UNIQUE;

-- ── Tags ───────────────────────────────────────────────────────
DEFINE TABLE tag SCHEMAFULL;
DEFINE FIELD name  ON tag TYPE string;
DEFINE FIELD slug  ON tag TYPE string;
DEFINE INDEX tag_slug ON tag COLUMNS slug UNIQUE;

-- ── Dangling link placeholders ─────────────────────────────────
DEFINE TABLE dangling_node SCHEMAFULL;
```

```
DEFINE FIELD name ON dangling_node TYPE string;
DEFINE INDEX dangling_name ON dangling_node COLUMNS name UNIQUE;

-- ── Relations ──────────────────────────────────────────────────────
DEFINE TABLE links_to SCHEMAFULL TYPE RELATION IN note OUT note | dangling_node;
DEFINE FIELD alias          ON links_to TYPE option<string>;
DEFINE FIELD section_anchor ON links_to TYPE option<string>;
DEFINE FIELD block_id       ON links_to TYPE option<string>;
DEFINE FIELD line_number    ON links_to TYPE int;

DEFINE TABLE tagged_with SCHEMAFULL TYPE RELATION IN note OUT tag;
DEFINE FIELD source ON tagged_with TYPE string;  -- 'inline' | 'frontmatter'

-- ── Vector chunks (Phase 8, defined early) ─────────────────────────
DEFINE TABLE embedding_chunk SCHEMAFULL;
DEFINE FIELD note_id     ON embedding_chunk TYPE record<note>;
DEFINE FIELD chunk_index ON embedding_chunk TYPE int;
DEFINE FIELD text        ON embedding_chunk TYPE string;
DEFINE FIELD vector      ON embedding_chunk TYPE array<float>;
```

Edge Case: *Schema DDL must be idempotent. SurrealDB's DEFINE ... is safe to re-run. Never use DROP TABLE in migrations — add new fields or tables only.*

3. Phase 1 — Core Vault Engine

Phase 1 establishes the foundation: opening a vault directory, scanning all Markdown files, parsing frontmatter, upserting records into SurrealDB, and exposing a note list and CRUD operations to the frontend. No wikilink parsing or graph edges are created in this phase.

3.1 Frontmatter and Markdown Parser

The parser is a pure Rust module with no side effects. It accepts raw file content as a string and returns a structured ParsedNote.

3.1.1 ParsedNote Struct

```
// src-tauri/src/engine/parser.rs
use std::collections::HashMap;

#[derive(Debug, Clone)]
pub struct ParsedNote {
    pub title: String,
    pub frontmatter: HashMap<String, serde_json::Value>,
    pub body: String,          // content after frontmatter block
    pub tags_inline: Vec<String>,   // from #tag syntax in body
    pub tags_frontmatter: Vec<String>, // from frontmatter tags: [] field
}
```

3.1.2 parse_note() Function

```
pub fn parse_note(path: &Path, content: &str) -> ParsedNote {
    let (frontmatter_raw, body) = split_frontmatter(content);
    let frontmatter = parse_frontmatter_yaml(frontmatter_raw);

    let title = extract_title(&frontmatter, body, path);
    let tags_frontmatter = extract_frontmatter_tags(&frontmatter);
    let tags_inline = extract_inline_tags(body);

    ParsedNote { title, frontmatter, body: body.to_string(),
                 tags_inline, tags_frontmatter }
}

/// Split off the leading --- ... --- block.
/// Returns (frontmatter_str, body_str). Both may be empty.
fn split_frontmatter(content: &str) -> (&str, &str) {
    if !content.starts_with("---") {
        return ("", content);
    }
    // Find closing --- after the opening line
    let after_open = &content[3..];
    let newline_pos = after_open.find('\n').map(|p| p + 1).unwrap_or(0);
    let search_region = &after_open[newline_pos..];
    if let Some(close) = search_region.find("\n---") {
        let fm_end = 3 + newline_pos + close;
        let body_start = fm_end + 4; // skip \n---
        // skip optional \n after closing ---
        let body_start = if content.as_bytes().get(body_start) == Some(&b'\n') {
            body_start + 1
        } else { body_start };
        (&content[3..fm_end], &content[body_start..])
    } else {
        ("", content)
    }
```

```
}

fn parse_frontmatter_yaml(raw: &str) -> HashMap<String, serde_json::Value> {
    if raw.is_empty() { return HashMap::new(); }
    serde_yaml::from_str(raw).unwrap_or_default()
}

fn extract_title(
    fm: &HashMap<String, serde_json::Value>,
    body: &str,
    path: &Path,
) -> String {
    // Priority 1: frontmatter `title` field
    if let Some(t) = fm.get("title").and_then(|v| v.as_str()) {
        return t.to_string();
    }
    // Priority 2: first H1 in body (# Title)
    for line in body.lines() {
        if let Some(h) = line.strip_prefix("# ") {
            return h.trim().to_string();
        }
    }
    // Priority 3: filename without extension
    path.file_stem()
        .and_then(|s| s.to_str())
        .unwrap_or("Untitled")
        .to_string()
}

fn extract_frontmatter_tags(
    fm: &HashMap<String, serde_json::Value>,
) -> Vec<String> {
    fm.get("tags")
      .and_then(|v| v.as_array())
      .map(|arr| arr.iter()
                    .filter_map(|v| v.as_str().map(|s| s.to_lowercase()))
                    .collect())
      .unwrap_or_default()
}

fn extract_inline_tags(body: &str) -> Vec<String> {
    // Match #word and #word/subword, not inside code blocks
    let re = regex::Regex::new(r"(?m)(?:^|\s)#([\w/]+)").unwrap();
    re.captures_iter(body)
      .map(|c| c[1].to_lowercase())
      .collect()
}
```

3.1.3 Edge Cases — Parser

Scenario	Expected Behaviour	Implementation Note
File has no frontmatter	Empty HashMap, full content is body	split_frontmatter returns ("", content)
Frontmatter YAML is malformed	Log warning, use empty HashMap; do not crash	unwrap_or_default() on serde_yaml result
File has only frontmatter, no body	body = empty string, title from frontmatter or filename	Handled naturally by split_frontmatter

Scenario	Expected Behaviour	Implementation Note
tags field is a string, not array	Treat as single-item tag list	Check as_str() fallback before as_array()
File is not valid UTF-8	Skip file, emit warning event to frontend	Handled in indexer before parse_note call
H1 contains formatting (# **Title**)	Strip Markdown bold/italics before using as title	Apply simple regex strip on the H1 line

3.2 Note ID Strategy

Note IDs are stable across machines and renames within the same vault. The ID is derived from the SHA-256 hash of the vault-relative path, hex-encoded.

```
use sha2::{Sha256, Digest};

pub fn note_id_from_path(vault_root: &Path, note_path: &Path) -> String {
    let relative = note_path
        .strip_prefix(vault_root)
        .expect("note path must be under vault root");
    // Normalize to forward-slash paths for cross-platform stability
    let normalized = relative.to_string_lossy().replace('\\', "/");
    let hash = Sha256::digest(normalized.as_bytes());
    format!("note:{}", hex::encode(&hash[..16]))  // 128-bit prefix is sufficient
}
```

Edge Case: *If a note is renamed, its ID changes. Cascade link refactoring (Phase 3, §5.3) must update all links_to edges pointing to the old ID before the rename completes.*

3.3 Vault Indexer

The indexer runs in a Tokio background task spawned by vault_open. It emits progress events to the frontend and populates SurrealDB.

3.3.1 IndexerService (src-tauri/src/engine/indexer.rs)

```
pub struct IndexerService {
    pub db: Arc<Surreal<SurrealKv>>,
    pub vault_root: PathBuf,
    pub app_handle: tauri::AppHandle,
}

impl IndexerService {
    pub async fn index_full(&self) -> Result<u64, IndexerError> {
        let paths = self.collect_md_paths().await?;
        let total = paths.len() as u64;

        for (i, path) in paths.iter().enumerate() {
            self.index_one(path).await?;
            let pct = ((i + 1) as f32 / total as f32 * 100.0) as u8;
            self.app_handle.emit("vault_index_progress", ProgressPayload {
                pct, message: format!("Indexed {}/{}", i + 1, total),
            }).ok();
        }
        Ok(total)
    }

    pub async fn index_one(&self, path: &Path) -> Result<(), IndexerError> {
```

```
        let content = tokio::fs::read_to_string(path).await
            .map_err(|e| IndexerError::ReadFailed(path.to_path_buf(), e))?;

        // Validate UTF-8 (read_to_string already does this)
        let parsed = parser::parse_note(path, &content);
        let id = note_id_from_path(&self.vault_root, path);

        db::notes::upsert(&self.db, &id, path, &parsed, &content).await?;
        db::notes::upsert_tags(&self.db, &id, &parsed).await?;
        // Wikilink edge creation is Phase 3
        Ok(())
    }

    async fn collect_md_paths(&self) -> Result<Vec<PathBuf>, IndexerError> {
        let mut result = Vec::new();
        let mut queue = vec![self.vault_root.clone()];
        while let Some(dir) = queue.pop() {
            let mut entries = tokio::fs::read_dir(&dir).await?;
            while let Some(entry) = entries.next_entry().await? {
                let path = entry.path();
                if path.is_dir() { queue.push(path); }
                else if path.extension().and_then(|e| e.to_str()) == Some("md") {
                    result.push(path);
                }
            }
        }
        Ok(result)
    }
}
```

3.3.2 IndexerError

```
#[derive(Debug, thiserror::Error)]
pub enum IndexerError {
    #[error("Failed to read {0}: {1}")]
    ReadFailed(PathBuf, std::io::Error),
    #[error("Database error: {0}")]
    DbError(#[from] surrealdb::Error),
    #[error("IO error: {0}")]
    Io(#[from] std::io::Error),
}

impl From<IndexerError> for String {
    fn from(e: IndexerError) -> Self { e.to_string() }
}
```

3.4 vault_open Command

3.4.1 Rust Handler (src-tauri/src/commands/vault.rs)

```
#[tauri::command]
pub async fn vault_open(
    path: String,
    state: tauri::State<'_, AppState>,
    app: tauri::AppHandle,
) -> Result<VaultInfo, String> {
    let vault_root = PathBuf::from(&path);

    // Validate the directory exists
    if !vault_root.is_dir() {
        return Err(format!("Path is not a directory: {}", path));
```

```
    }

    // Stop existing watcher if any
    {
        let mut w = state.watcher.write().await;
        *w = None; // drop triggers notify cleanup
    }

    // Update vault path in state
    {
        let mut vp = state.vault_path.write().await;
        *vp = Some(vault_root.clone());
    }

    // Persist last-opened path
    // tauri-plugin-store save is called here (see §3.4.2)

    // Spawn background indexer
    let db = state.db.clone();
    let app_handle = app.clone();
    let root = vault_root.clone();
    tokio::spawn(async move {
        let indexer = IndexerService { db, vault_root: root.clone(), app_handle:
app_handle.clone() };
        match indexer.index_full().await {
            Ok(count) => {
                app_handle.emit("vault_index_done", count).ok();
                // Start file watcher after index completes
                watcher::start(&app_handle, &root).await.ok();
            }
            Err(e) => {
                app_handle.emit("vault_index_error", e.to_string()).ok();
            }
        }
    });

    // Return immediately; progress comes via events
    Ok(VaultInfo { path, note_count: 0 })
}
```

3.4.2 TypeScript Caller (src/hooks/useVault.ts)

```
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';
import { open } from '@tauri-apps/plugin-dialog';
import type { VaultInfo, ProgressPayload } from '../types/ipc';

export function useVault() {
  const openVault = async () => {
    const selected = await open({ directory: true, multiple: false });
    if (!selected || Array.isArray(selected)) return;

    // Listen for progress before invoking
    const unlistenProgress = await listen<ProgressPayload>(
      'vault_index_progress',
      (event) => setIndexProgress(event.payload.pct)
    );
    const unlistenDone = await listen<number>(
      'vault_index_done',
      (event) => { setNoteCount(event.payload); unlistenProgress(); unlistenDone();
}
```

```
    );

    await invoke<VaultInfo>('vault_open', { path: selected });
  };

  return { openVault };
}
```

3.4.3 Edge Cases — vault_open

Scenario	Handling
Path does not exist	Return Err immediately; frontend shows toast
Path is a file, not directory	Return Err; frontend shows toast
Vault contains 0 .md files	Emit vault_index_done with count=0; show empty state in UI
Vault re-opened while indexing in progress	Write lock on watcher drops old watcher; old Tokio task may still be running — emit from old task is harmless as frontend re-attaches listeners
File read fails mid-index (permissions)	Log error, skip file, continue indexing; emit warning event
Very large vault (>10k notes)	Progress events must fire at least every 100 files to keep UI responsive

3.5 Note CRUD Commands

3.5.1 note_list

```
/// Returns all notes sorted by updated_at descending
#[tauri::command]
pub async fn note_list(
    state: tauri::State<'_, AppState>,
) -> Result<Vec<NoteSummary>, String> {
    let results: Vec<NoteSummary> = state.db
        .query("SELECT id, path, title, updated_at, tags FROM note ORDER BY
updated_at DESC")
        .await.map_err(|e| e.to_string())?
        .take(0).map_err(|e| e.to_string())?;
    Ok(results)
}
```

3.5.2 note_read

```
/// Reads note content from DISK (source of truth), not from DB.
/// Returns full NoteRecord including freshly-read content.
#[tauri::command]
pub async fn note_read(
    path: String,
    state: tauri::State<'_, AppState>,
) -> Result<NoteRecord, String> {
    let p = PathBuf::from(&path);
    let content = tokio::fs::read_to_string(&p).await
        .map_err(|e| format!("Failed to read {}: {}", path, e))?;

    // Also fetch metadata from DB (created_at, id)
    let meta: Option<NoteRecord> = state.db
```

```
        .query("SELECT id, path, title, frontmatter, created_at, updated_at FROM
note WHERE path = $path")
        .bind(("path", &path))
        .await.map_err(|e| e.to_string())?
        .take(0).map_err(|e| e.to_string())?;

    let mut record = meta.ok_or_else(|| format!("Note not found in index: {}",
path))?;
    record.content = content; // Always use disk content
    Ok(record)
}
```

3.5.3 note_save

```
/// Writes content to disk, then re-parses and updates the SurrealDB record.
#[tauri::command]
pub async fn note_save(
    path: String,
    content: String,
    state: tauri::State<'_, AppState>,
    app: tauri::AppHandle,
) -> Result<NoteSummary, String> {
    // 1. Write to disk
    tokio::fs::write(&path, &content).await
        .map_err(|e| format!("Write failed: {}", e))?;

    // 2. Re-parse
    let p = PathBuf::from(&path);
    let vault_path = state.vault_path.read().await
        .clone().ok_or("No vault open")?;
    let parsed = parser::parse_note(&p, &content);
    let id = note_id_from_path(&vault_path, &p);

    // 3. Upsert DB record
    let summary = db::notes::upsert(&state.db, &id, &p, &parsed, &content).await
        .map_err(|e| e.to_string())?;

    // 4. Update tags
    db::notes::upsert_tags(&state.db, &id, &parsed).await
        .map_err(|e| e.to_string())?;

    // Note: wikilink re-parsing is Phase 3
    Ok(summary)
}
```

3.5.4 note_create

```
#[tauri::command]
pub async fn note_create(
    title: String,
    state: tauri::State<'_, AppState>,
) -> Result<NoteSummary, String> {
    let vault_root = state.vault_path.read().await
        .clone().ok_or("No vault open")?;

    // Sanitize title to filesystem-safe filename
    let safe_name = sanitize_filename(&title);
    let mut path = vault_root.join(format!("{}.md", safe_name));

    // Avoid collisions: append _2, _3, etc.
    let mut counter = 2u32;
    while path.exists() {
```

```
        path = vault_root.join(format!("{}_{}.md", safe_name, counter));
        counter += 1;
    }

    let frontmatter = format!("---\ntitle: {}\ndate: {}\n---\n",
        title,
        chrono::Local::now().format("%Y-%m-%d"),
    );
    tokio::fs::write(&path, &frontmatter).await
        .map_err(|e| e.to_string())?;

    // Index the new note
    let indexer = IndexerService {
        db: state.db.clone(),
        vault_root: vault_root.clone(),
        app_handle: /* passed via extra arg */ todo!(),
    };
    indexer.index_one(&path).await.map_err(|e| e.to_string())?;

    // Return summary
    let id = note_id_from_path(&vault_root, &path);
    db::notes::get_summary(&state.db, &id).await.map_err(|e| e.to_string())
}

fn sanitize_filename(title: &str) -> String {
    // Remove characters unsafe on any target OS (Windows is strictest)
    let bad: &[char] = &['/', '\\', ':', '*', '?', '"', '<', '>', '|'];
    title.chars().filter(|c|
!bad.contains(c)).collect::<String>().trim().to_string()
}
```

3.5.5 note_delete

```
#[tauri::command]
pub async fn note_delete(
    path: String,
    state: tauri::State<'_, AppState>,
) -> Result<(), String> {
    let vault_root = state.vault_path.read().await.clone().ok_or("No vault open")?;
    let p = PathBuf::from(&path);
    let id = note_id_from_path(&vault_root, &p);

    // 1. Delete from disk
    tokio::fs::remove_file(&p).await.map_err(|e| e.to_string())?;

    // 2. Remove DB record (SurrealDB cascades edge deletion when node is deleted)
    state.db.query("DELETE note WHERE id = $id")
        .bind(("id", &id))
        .await.map_err(|e| e.to_string())?;

    // 3. Convert outgoing links to dangling_nodes (Phase 3 concern)
    //    For now, just delete all links_to edges involving this note
    state.db.query(
        "DELETE links_to WHERE in = $id OR out = $id; DELETE tagged_with WHERE in =
$id"
    ).bind(("id", &id)).await.map_err(|e| e.to_string())?;

    Ok(())
}
```

3.5.6 Edge Cases — Note CRUD

Scenario	Handling
note_save on a path that no longer exists on disk	tokio::fs::write creates the file; this handles the case where file was deleted externally
note_create: title contains only special characters	sanitize_filename returns empty string → fall back to 'Untitled'
note_create: filename collision after sanitizing two different titles	Counter suffix ensures uniqueness (_2, _3, ...)
note_delete: file already deleted externally	Ignore ENOENT from fs::remove_file; still clean up DB
note_read: note in DB but file missing from disk	Return Err; frontend shows 'File not found' toast
note_list with no vault open	Return empty Vec (no error) — empty state is valid

3.6 File Watcher

After initial indexing completes, a notify file watcher monitors the vault directory for external changes.

3.6.1 Watcher Setup (src-tauri/src/engine/watcher.rs)

```
use notify::{Watcher, RecursiveMode, RecommendedWatcher, Event, EventKind};
use tokio::sync::mpsc;

pub async fn start(
    app: &tauri::AppHandle,
    vault_root: &PathBuf,
    state: tauri::State<'_, AppState>,
) -> Result<(), notify::Error> {
    let (tx, mut rx) = mpsc::channel::<notify::Result<Event>>(32);
    let app_handle = app.clone();
    let db = state.db.clone();
    let root = vault_root.clone();

    let mut watcher = RecommendedWatcher::new(
        move |res| { let _ = tx.blocking_send(res); },
        notify::Config::default(),
    )?;
    watcher.watch(vault_root, RecursiveMode::Recursive)?;

    // Store watcher in state to keep it alive
    *state.watcher.write().await = Some(watcher);

    // Background task: process events
    tokio::spawn(async move {
        while let Some(Ok(event)) = rx.recv().await {
            match event.kind {
                EventKind::Create(_) | EventKind::Modify(_) => {
                    for path in event.paths.iter().filter(|p| p.extension()
                        .and_then(|e| e.to_str()) == Some("md")) {
                        let indexer = IndexerService {
                            db: db.clone(), vault_root: root.clone(),
```

```
                            app_handle: app_handle.clone(),
                        };
                        indexer.index_one(path).await.ok();
                    }
                }
                EventKind::Remove(_) => {
                    // Remove from DB — path is gone, use it to find the record
                    for path in &event.paths {
                        let path_str = path.to_string_lossy().to_string();
                        db.query("DELETE note WHERE path = $p")
                            .bind(("p", path_str)).await.ok();
                    }
                }
                _ => {} // Ignore access events
            }
        }
    });
    Ok(())
}
```

Android Note: *notify's RecommendedWatcher may fall back to polling on Android. This is acceptable but must be validated in Phase 9. See Architecture §4.3.*

4. Phase 2 — Editor and Rendering

Phase 2 delivers the user-facing editing experience: the CodeMirror 6 editor, the live Markdown preview pane, KaTeX math rendering, and the theme system. All work in this phase is frontend-only.

4.1 CodeMirror 6 Integration

CodeMirror 6 (CM6) manages its own DOM subtree. The integration pattern is: React holds the note content string as state; CM6 is mounted once per note and listens for changes; on note switch, CM6 is re-initialized with the new content.

4.1.1 Editor.tsx Component

```
// src/components/Editor.tsx
import { useEffect, useRef } from 'react';
import { EditorView, keymap, lineNumbers } from '@codemirror/view';
import { EditorState } from '@codemirror/state';
import { defaultKeymap, historyKeymap, history } from '@codemirror/commands';
import { markdown } from '@codemirror/lang-markdown';
import { languages } from '@codemirror/language-data';
import { oneDark } from '@codemirror/theme-one-dark';

interface EditorProps {
  content: string;              // Controlled from parent (React state)
  onChange: (val: string) => void;
  noteId: string;               // Used to detect note switches
  isDark: boolean;
}

export function Editor({ content, onChange, noteId, isDark }: EditorProps) {
  const containerRef = useRef<HTMLDivElement>(null);
  const viewRef = useRef<EditorView | null>(null);

  // Re-create EditorView when note switches
  useEffect(() => {
    if (!containerRef.current) return;
    viewRef.current?.destroy();

    const state = EditorState.create({
      doc: content,
      extensions: [
        history(),
        lineNumbers(),
        markdown({ codeLanguages: languages }),
        keymap.of([...defaultKeymap, ...historyKeymap]),
        isDark ? oneDark : [],
        EditorView.updateListener.of((update) => {
          if (update.docChanged) {
            onChange(update.state.doc.toString());
          }
        }),
        EditorView.theme({
          '&': { height: '100%', fontSize: '14px' },
          '.cm-scroller': { overflow: 'auto', fontFamily: 'var(--font-mono)' },
        }),
      ],
    });
```

```
    viewRef.current = new EditorView({ state, parent: containerRef.current });

    return () => viewRef.current?.destroy();
  // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [noteId]); // Only re-create when note changes, NOT on content or isDark

  // Sync isDark changes without re-creating
  useEffect(() => {
    // Dispatch a config transaction — CM6 supports dynamic theme swapping
    // Implementation: use a StateField for theme; dispatch reconfigure()
  }, [isDark]);

  return <div ref={containerRef} style={{ height: '100%', width: '100%' }} />;
}
```

4.1.2 Ownership Rules — CM6 and React

Concern	Rule
Who owns keystrokes?	CM6 owns all typing. React never writes to CM6's internal doc after mount.
How does React get content?	Via EditorView.updateListener.of() callback → call onChange → React setState.
How does React set content?	Never directly. On note switch, destroy and re-create EditorView with new doc.
Programmatic inserts (templates)?	Use viewRef.current.dispatch({ changes: { from, to, insert } }) — never setState + re-create.
Save timing	Debounce onChange handler by 500ms before calling note_save; avoids excessive IPC calls.

4.2 Markdown Preview (Preview.tsx)

The preview pane renders the editor content as HTML using a unified/remark pipeline, with KaTeX handling math tokens before the HTML is output.

4.2.1 Render Pipeline

```
// src/components/Preview.tsx
import { useMemo } from 'react';
import { unified } from 'unified';
import remarkParse from 'remark-parse';
import remarkGfm from 'remark-gfm';
import remarkMath from 'remark-math';
import remarkRehype from 'remark-rehype';
import rehypeKatex from 'rehype-katex';
import rehypeHighlight from 'rehype-highlight';
import rehypeStringify from 'rehype-stringify';
import 'katex/dist/katex.min.css';

const processor = unified()
  .use(remarkParse)        // parse Markdown
  .use(remarkGfm)          // GitHub-flavoured Markdown (tables, task lists)
  .use(remarkMath)         // $...$ and $$...$$ math nodes
  .use(remarkRehype, { allowDangerousHtml: false })
  .use(rehypeKatex)        // render math nodes with KaTeX
  .use(rehypeHighlight)    // syntax-highlight fenced code blocks
```

```
  .use(rehypeStringify);

interface PreviewProps { content: string; }

export function Preview({ content }: PreviewProps) {
  const html = useMemo(() => {
    try {
      return String(processor.processSync(content));
    } catch {
      return '<p style="color:red">Render error</p>';
    }
  }, [content]);

  return (
    <div
      className="prose dark:prose-invert max-w-none p-4 overflow-auto h-full"
      dangerouslySetInnerHTML={{ __html: html }}
    />
  );
}
```

Security Note: *dangerouslySetInnerHTML is acceptable here because the content is the user's own Markdown files from their local disk. No external HTML is ever injected. rehype-sanitize can be added as a precaution if needed.*

4.2.2 Split-Pane Layout

```
// src/App.tsx (simplified layout)
type PaneMode = 'editor' | 'preview' | 'split';

function NoteArea({ note, mode }: { note: NoteRecord; mode: PaneMode }) {
  const [content, setContent] = useState(note.content);
  const debouncedSave = useDebouncedCallback(
    (val: string) => invoke('note_save', { path: note.path, content: val }),
    500
  );

  return (
    <div style={{ display: 'grid', gridTemplateColumns: mode === 'split' ? '1fr
1fr' : '1fr', height: '100%' }}>
      {(mode === 'editor' || mode === 'split') && (
        <Editor content={content} onChange={(v) => { setContent(v);
debouncedSave(v); }}
                noteId={note.id} isDark={isDark} />
      )}
      {(mode === 'preview' || mode === 'split') && (
        <Preview content={content} />
      )}
    </div>
  );
}
```

4.3 Theme System

4.3.1 CSS Custom Properties

```
/* src/styles/theme.css */
:root {
  --color-bg:         #ffffff;
  --color-surface:    #f5f5f5;
  --color-border:     #e0e0e0;
```

```
  --color-text:       #1a1a1a;
  --color-text-muted: #666666;
  --color-accent:     #2E6CA4;
  --font-mono:        'JetBrains Mono', 'Fira Code', monospace;
}

.dark {
  --color-bg:         #1e1e2e;
  --color-surface:    #2a2a3e;
  --color-border:     #3a3a5c;
  --color-text:       #cdd6f4;
  --color-text-muted: #6c7086;
  --color-accent:     #89b4fa;
}
```

4.3.2 useTheme Hook

```
// src/hooks/useTheme.ts
import { useEffect, useState } from 'react';
import { load as loadStore } from '@tauri-apps/plugin-store';

export function useTheme() {
  const systemDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
  const [isDark, setIsDark] = useState(systemDark);

  useEffect(() => {
    // Load persisted preference
    loadStore('settings.json').then(store =>
      store.get<boolean>('dark_mode').then(v => {
        if (v !== null) setIsDark(v);
      })
    );
  }, []);

  useEffect(() => {
    document.documentElement.classList.toggle('dark', isDark);
  }, [isDark]);

  const toggleTheme = async () => {
    const next = !isDark;
    setIsDark(next);
    const store = await loadStore('settings.json');
    await store.set('dark_mode', next);
    await store.save();
  };

  return { isDark, toggleTheme };
}
```

5. Phase 3 — Linking Engine

Phase 3 is the defining feature of GraphNotes. It adds wikilink parsing, graph edge creation, the backlinks panel, dangling link handling, cascade rename, and transclusion rendering.

5.1 Wikilink Parser

The wikilink parser runs as part of the indexer (called from index_one) and also on every note_save. It is a pure function operating on a content string.

5.1.1 WikilinkMatch Struct

```
// src-tauri/src/engine/parser.rs  (extended)
#[derive(Debug, Clone)]
pub struct WikilinkMatch {
    pub target: String,          // Note title or path (before #)
    pub alias: Option<String>,   // Display text after |
    pub section_anchor: Option<String>,  // After # (heading)
    pub block_id: Option<String>,        // After #^ (block ref)
    pub is_transclusion: bool,   // True if preceded by !
    pub line_number: u32,
}
```

5.1.2 extract_wikilinks()

```
pub fn extract_wikilinks(content: &str) -> Vec<WikilinkMatch> {
    // Match: optional ! prefix, [[, target, optional |alias or #anchor, ]]
    // Regex breakdown:
    //  (!?)       — capture optional transclusion prefix
    //  \[\[       — opening brackets
    //  ([^\]#|]+) — target name (no ], #, |)
    //  (#\^?([^\]|]+))? — optional #section or #^blockid
    //  (\|([^\]]+))? — optional |alias
    //  \]\]       — closing brackets
    let re = regex::Regex::new(
        r"(?m)(!?)\[\[([^\]#|]+?)(?:#(\^?)([^\]|]+?))?(?:\|([^\]]+?))?\]\]"
    ).unwrap();

    let mut results = Vec::new();
    let mut in_code_block = false;

    for (line_no, line) in content.lines().enumerate() {
        // Skip content inside fenced code blocks
        if line.trim_start().starts_with("```") {
            in_code_block = !in_code_block;
            continue;
        }
        if in_code_block { continue; }

        for cap in re.captures_iter(line) {
            let is_transclusion = &cap[1] == "!";
            let target = cap[2].trim().to_string();
            let block_prefix = cap.get(3).map(|m| m.as_str()).unwrap_or("");
            let anchor_or_block = cap.get(4).map(|m| m.as_str().to_string());
            let alias = cap.get(5).map(|m| m.as_str().to_string());

            let (section_anchor, block_id) = if block_prefix == "^" {
                (None, anchor_or_block)
            } else {
                (anchor_or_block, None)
```

```
            };

            results.push(WikilinkMatch {
                target,
                alias,
                section_anchor,
                block_id,
                is_transclusion,
                line_number: (line_no + 1) as u32,
            });
        }
    }
    results
}
```

5.1.3 Wikilink Edge Cases

Input	Expected Output
[[Note Name]]	target='Note Name', alias=None, anchor=None
[[Note Name|Display Text]]	target='Note Name', alias=Some('Display Text')
[[Note Name#Section Heading]]	target='Note Name', section_anchor=Some('Section Heading')
[[Note Name#^block-123]]	target='Note Name', block_id=Some('block-123')
![[Embedded Note]]	target='Embedded Note', is_transclusion=true
[[Note]] inside ```code block```	Not matched — skipped
[[Note]] inside `inline code`	Still matched — inline code not tracked at line level
[[Target|]] (empty alias)	alias treated as None
[[]] (empty target)	Skipped — regex requires ≥1 char in target
[[Note with spaces]]	target='Note with spaces' — spaces are valid

5.2 Link Resolution and Edge Creation

After extracting wikilinks, the indexer resolves each target to a note record or creates a dangling_node, then writes links_to edges in SurrealDB.

5.2.1 resolve_and_upsert_links()

```
// src-tauri/src/engine/indexer.rs  (extended)
pub async fn resolve_and_upsert_links(
    db: &Surreal<SurrealKv>,
    source_id: &str,
    wikilinks: &[WikilinkMatch],
) -> Result<(), surrealdb::Error> {
    // Delete existing links FROM this source (full re-index on every save)
    db.query("DELETE links_to WHERE in = $src")
        .bind(("src", source_id))
        .await?;

    for wl in wikilinks {
        let target_id = resolve_target(db, &wl.target).await?;
        let (out_table, out_id) = match target_id {
            Some(id) => ("note", id),
```

```
            None => {
                // Upsert dangling_node
                let dn: Vec<serde_json::Value> = db.query(
                    "INSERT INTO dangling_node (name) VALUES ($name) ON DUPLICATE
KEY IGNORE"
                ).bind(("name", &wl.target)).await?.take(0)?;
                let dn_id = dn.first()
                    .and_then(|v| v.get("id"))
                    .and_then(|v| v.as_str())
                    .unwrap_or_default().to_string();
                ("dangling_node", dn_id)
            }
        };

        db.query(
            "RELATE $src -> links_to -> $tgt CONTENT {
                alias: $alias,
                section_anchor: $section,
                block_id: $block,
                line_number: $line
            }"
        )
        .bind(("src", source_id))
        .bind(("tgt", &format!("{out_table}:{out_id}")))
        .bind(("alias", &wl.alias))
        .bind(("section", &wl.section_anchor))
        .bind(("block", &wl.block_id))
        .bind(("line", wl.line_number))
        .await?;
    }
    Ok(())
}

/// Case-insensitive title lookup; returns note ID if found.
async fn resolve_target(
    db: &Surreal<SurrealKv>,
    target: &str,
) -> Result<Option<String>, surrealdb::Error> {
    let results: Vec<serde_json::Value> = db.query(
        "SELECT id FROM note WHERE string::lowercase(title) = string::lowercase($t)
LIMIT 1"
    ).bind(("t", target)).await?.take(0)?;
    Ok(results.first()
        .and_then(|v| v.get("id"))
        .and_then(|v| v.as_str())
        .map(|s| s.to_string()))
}
```

5.3 graph_query_backlinks Command

```
#[tauri::command]
pub async fn graph_query_backlinks(
    path: String,
    state: tauri::State<'_, AppState>,
) -> Result<Vec<BacklinkEntry>, String> {
    // Traverse INCOMING links_to edges to this note
    let results: Vec<serde_json::Value> = state.db.query(
        "SELECT in.path AS source_path, in.title AS source_title, in.content AS
source_content
         FROM links_to WHERE out.path = $path"
    ).bind(("path", &path))
     .await.map_err(|e| e.to_string())?
```

```
     .take(0).map_err(|e| e.to_string())?;

    let entries = results.into_iter().map(|row| {
        let source_path = row["source_path"].as_str().unwrap_or("").to_string();
        let source_title = row["source_title"].as_str().unwrap_or("").to_string();
        let content = row["source_content"].as_str().unwrap_or("");
        let snippet = extract_snippet(content, &path);
        BacklinkEntry { source_path, source_title, snippet }
    }).collect();

    Ok(entries)
}

/// Extract 1-2 lines around the wikilink occurrence in source_content.
fn extract_snippet(content: &str, target_title: &str) -> String {
    let target_lower = target_title.to_lowercase();
    for (i, line) in content.lines().enumerate() {
        if line.to_lowercase().contains(&target_lower) {
            let start = i.saturating_sub(1);
            let end = (i + 2).min(content.lines().count());
            return content.lines().skip(start).take(end - start)
                          .collect::<Vec<_>>().join(" ");
        }
    }
    String::new()
}
```

5.4 graph_query_unlinked_mentions Command

```
#[tauri::command]
pub async fn graph_query_unlinked_mentions(
    path: String,
    title: String,
    state: tauri::State<'_, AppState>,
) -> Result<Vec<BacklinkEntry>, String> {
    // Full-text search for title string in all notes,
    // excluding notes that already have a links_to edge to this note.
    let results: Vec<serde_json::Value> = state.db.query(
        "SELECT id, path, title, content FROM note
         WHERE path != $path
           AND string::contains(string::lowercase(content),
string::lowercase($title))
           AND id NOT IN (
             SELECT in FROM links_to WHERE out.path = $path
           )"
    ).bind(("path", &path)).bind(("title", &title))
     .await.map_err(|e| e.to_string())?
     .take(0).map_err(|e| e.to_string())?;

    let entries = results.into_iter().map(|row| BacklinkEntry {
        source_path: row["path"].as_str().unwrap_or("").to_string(),
        source_title: row["title"].as_str().unwrap_or("").to_string(),
        snippet: extract_snippet(
            row["content"].as_str().unwrap_or(""),
            &title
        ),
    }).collect();
    Ok(entries)
}
```

Performance Note: *The unlinked mentions query uses string::contains which is O(n) across all note content. For vaults over 5,000 notes this may be slow. Phase 5 (full-text search index) provides a*

proper solution via SEARCH ANALYZER. For Phase 3, the naïve implementation is acceptable.

5.5 BacklinksPanel.tsx

```
// src/components/BacklinksPanel.tsx
import { useEffect, useState } from 'react';
import { invoke } from '@tauri-apps/api/core';
import type { BacklinkEntry, NoteRecord } from '../types/ipc';

interface Props { note: NoteRecord; onNavigate: (path: string) => void; }

export function BacklinksPanel({ note, onNavigate }: Props) {
  const [backlinks, setBacklinks] = useState<BacklinkEntry[]>([]);
  const [unlinked, setUnlinked] = useState<BacklinkEntry[]>([]);

  useEffect(() => {
    invoke<BacklinkEntry[]>('graph_query_backlinks', { path: note.path })
      .then(setBacklinks);
    invoke<BacklinkEntry[]>('graph_query_unlinked_mentions',
      { path: note.path, title: note.title })
      .then(setUnlinked);
  }, [note.id]);

  return (
    <div className="backlinks-panel">
      <section>
        <h3>Linked Mentions ({backlinks.length})</h3>
        {backlinks.map(bl => (
          <div key={bl.source_path} onClick={() => onNavigate(bl.source_path)}
               className="backlink-entry">
            <strong>{bl.source_title}</strong>
            <p className="snippet">{bl.snippet}</p>
          </div>
        ))}
      </section>
      <section>
        <h3>Unlinked Mentions ({unlinked.length})</h3>
        {unlinked.map(ul => (
          <div key={ul.source_path} onClick={() => onNavigate(ul.source_path)}
               className="backlink-entry unlinked">
            <strong>{ul.source_title}</strong>
            <p className="snippet">{ul.snippet}</p>
          </div>
        ))}
      </section>
    </div>
  );
}
```

5.6 Cascade Link Refactoring

When a note is renamed, all wikilinks across the vault pointing to the old title must be updated. This runs as a Tokio background task.

5.6.1 note_rename Command

```
#[tauri::command]
pub async fn note_rename(
    old_path: String,
    new_title: String,
```

```
    state: tauri::State<'_, AppState>,
    app: tauri::AppHandle,
) -> Result<NoteSummary, String> {
    let vault_root = state.vault_path.read().await.clone().ok_or("No vault open")?;
    let old_p = PathBuf::from(&old_path);
    let old_title = old_p.file_stem()
        .and_then(|s| s.to_str()).unwrap_or("").to_string();

    // Build new file path
    let new_filename = sanitize_filename(&new_title);
    let new_p = old_p.with_file_name(format!("{}.md", new_filename));
    if new_p.exists() {
        return Err(format!("A note named '{}' already exists.", new_title));
    }

    // 1. Rename file on disk
    tokio::fs::rename(&old_p, &new_p).await.map_err(|e| e.to_string())?;

    // 2. Update DB record: path + title + id
    let old_id = note_id_from_path(&vault_root, &old_p);
    let new_id = note_id_from_path(&vault_root, &new_p);
    state.db.query(
        "UPDATE note SET path = $new_path, title = $new_title WHERE id = $old_id"
    ).bind(("new_path", new_p.to_string_lossy().to_string()))
     .bind(("new_title", &new_title))
     .bind(("old_id", &old_id))
     .await.map_err(|e| e.to_string())?;

    // 3. Spawn cascade refactor in background
    let db = state.db.clone();
    let app_h = app.clone();
    let old_t = old_title.clone();
    let new_t = new_title.clone();
    let root = vault_root.clone();
    tokio::spawn(async move {
        let count = refactor::cascade_rename(&db, &root, &old_t, &new_t,
&app_h).await
            .unwrap_or(0);
        app_h.emit("refactor_done", RefactorResult { files_updated: count }).ok();
    });

    db::notes::get_summary(&state.db, &new_id).await.map_err(|e| e.to_string())
}
```

5.6.2 cascade_rename() (src-tauri/src/engine/refactor.rs)

```
pub async fn cascade_rename(
    db: &Surreal<SurrealKv>,
    vault_root: &Path,
    old_title: &str,
    new_title: &str,
    app: &tauri::AppHandle,
) -> Result<u32, Box<dyn std::error::Error + Send + Sync>> {
    // Find all notes with links_to edges pointing to the old title
    let sources: Vec<serde_json::Value> = db.query(
        "SELECT in.path AS path FROM links_to WHERE out.title = $old"
    ).bind(("old", old_title)).await?.take(0)?;

    let total = sources.len() as u32;
    let mut updated = 0u32;
```

```
    // Regex patterns to match all wikilink variants referencing old_title
    // Matches: [[Old Title]], [[Old Title|alias]], [[Old Title#Section]],
    //          [[Old Title#^block]], ![[Old Title]]
    let pattern = format!(
        r"(!?)\[\[{}((?:#[^\]]*)?(?:\|[^\]]*)?)\]\]",
        regex::escape(old_title)
    );
    let re = regex::Regex::new(&pattern)?;

    for (i, src) in sources.iter().enumerate() {
        let path_str = src["path"].as_str().unwrap_or("");
        let path = PathBuf::from(path_str);

        let content = tokio::fs::read_to_string(&path).await?;
        let new_content = re.replace_all(&content, |caps: &regex::Captures| {
            let prefix = &caps[1];   // '!' or ''
            let suffix = &caps[2];   // '#Section' or '|alias' or ''
            format!("{}[[{}{}]]", prefix, new_title, suffix)
        }).to_string();

        if new_content != content {
            tokio::fs::write(&path, &new_content).await?;
            updated += 1;
        }

        let pct = ((i + 1) as f32 / total as f32 * 100.0) as u8;
        app.emit("refactor_progress", ProgressPayload { pct,
            message: format!("Updated {}/{} files", updated, total) }).ok();
    }
    Ok(updated)
}
```

5.6.3 Cascade Refactor Edge Cases

Scenario	Handling
Two notes with the same title	resolve_target returns first match; warn in log. This is a data quality issue the user must resolve.
Old title appears as plain text (not wikilink)	Not touched by cascade_rename. Unlinked mentions are display-only.
Source file has been deleted externally by the time refactor runs	read_to_string fails; log error, skip file, continue. Still emit refactor_done.
New title conflicts with an existing note title	note_rename checks new_p.exists() and returns Err before spawning the task.
Wikilink with alias: [[Old Title|My Alias]]	Regex preserves suffix: becomes [[New Title|My Alias]]. Alias is not changed.
Wikilink with section: [[Old Title#Heading]]	Regex preserves suffix: becomes [[New Title#Heading]].

5.7 Transclusion Rendering

Transclusion (![[Note Name]]) is handled entirely in the frontend preview renderer. When the HTML pipeline encounters a transclusion token, it fetches the target note's content via

note_read and renders it inline.

5.7.1 Transclusion in the Rehype Pipeline

```
// src/lib/remarkTransclusion.ts
// A custom remark plugin that transforms ![[Target]] into a placeholder node,
// then a rehype plugin replaces the placeholder with fetched note HTML.

import { invoke } from '@tauri-apps/api/core';
import type { NoteRecord } from '../types/ipc';

// Because fetching is async and remark/rehype run sync,
// we pre-fetch all transclusion targets before running the pipeline.

export async function prefetchTransclusions(
  content: string
): Promise<Map<string, string>> {
  const transclusionRe = /!\[\[([^\]#|]+?)(?:#([^\]|]+?))?\]\]/g;
  const fetches = new Map<string, Promise<string>>();

  for (const match of content.matchAll(transclusionRe)) {
    const target = match[1].trim();
    if (!fetches.has(target)) {
      fetches.set(target, invoke<NoteRecord>('note_read', { path: target })
        .then(r => r.content)
        .catch(() => `> Note not found: ${target}`));
    }
  }

  const resolved = new Map<string, string>();
  for (const [key, promise] of fetches) {
    resolved.set(key, await promise);
  }
  return resolved;
}

// Then in Preview.tsx, before running the processor:
// const transclusions = await prefetchTransclusions(content);
// const expandedContent = content.replace(
//   /!\[\[([^\]]+?)\]\]/g,
//   (_, target) => transclusions.get(target.trim()) ?? `> Note not found:
${target}`
// );
```

Edge Case: *Circular transclusions (Note A embeds Note B which embeds Note A) must be detected and broken. Track a Set of in-progress paths during prefetch; if target is already in the set, replace with an error message rather than recursing.*

6. IPC Command Reference (Phases 1–3)

Complete reference table for all Tauri commands exposed in Phases 1–3.

Command	Parameters	Return Type	Phase
vault_open	path: String	Result<VaultInfo, String>	1
vault_close	(none)	Result<(), String>	1
vault_rebuild	(none)	Result<(), String> (async via events)	1
note_list	(none)	Result<Vec<NoteSummary>, String>	1
note_read	path: String	Result<NoteRecord, String>	1
note_save	path: String, content: String	Result<NoteSummary, String>	1
note_create	title: String	Result<NoteSummary, String>	1
note_delete	path: String	Result<(), String>	1
note_rename	old_path: String, new_title: String	Result<NoteSummary, String>	3
graph_query_backlinks	path: String	Result<Vec<BacklinkEntry>, String>	3
graph_query_forward_links	path: String	Result<Vec<NoteSummary>, String>	3
graph_query_unlinked_mentions	path: String, title: String	Result<Vec<BacklinkEntry>, String>	3

6.1 Tauri Events (Backend → Frontend)

Event Name	Payload / Purpose
vault_index_progress	ProgressPayload { pct: u8, message: String } — emitted during initial vault scan
vault_index_done	u64 — total note count after indexing completes
vault_index_error	String — error message if indexing fails
refactor_progress	ProgressPayload — emitted per file during cascade rename
refactor_done	RefactorResult { files_updated: u32 }
file_changed	String (path) — emitted by file watcher when an external edit is detected

7. SRS Requirement Coverage

Traceability matrix mapping SRS v2.0 requirements to the implementing components in this TDD.

Req ID	Requirement (Short)	Implementing Component	Section
FR-001	Open directory as vault	vault_open command + directory picker	§3.4
FR-002	Notes as UTF-8 .md files	note_save writes .md only; indexer skips non-.md	§3.5.3
FR-003	File change watching	notify watcher → indexer.index_one()	§3.6
FR-004	Nested subdirectory support	IndexerService.collect_md_paths() recursive	§3.3.1
FR-005	YAML frontmatter extraction	parse_note() → split_frontmatter() + parse_frontmatter_yaml()	§3.1.2
FR-010	Wikilink edge creation	extract_wikilinks() + resolve_and_upsert_links()	§5.1–5.2
FR-011	Case-insensitive link resolution	resolve_target() using string::lowercase()	§5.2.1
FR-012	Link aliases	WikilinkMatch.alias field; stored on links_to edge	§5.1.1
FR-013	Dangling link tracking	dangling_node upsert in resolve_and_upsert_links()	§5.2.1
FR-014	Dangling link → create note	Frontend onClick → note_create (BacklinksPanel)	§5.5
FR-015	Backlinks panel	graph_query_backlinks + BacklinksPanel.tsx	§5.3, §5.5
FR-016	Unlinked mentions	graph_query_unlinked_mentions	§5.4
FR-017	Backlink context snippet	extract_snippet() in graph_query_backlinks	§5.3
FR-018	Cascade link refactoring on rename	note_rename + cascade_rename()	§5.6
FR-019	Cascade refactor as background task	tokio::spawn in note_rename	§5.6.1
FR-020	Section links [[Note#Heading]]	WikilinkMatch.section_anchor; stored on links_to edge	§5.1.1
FR-021	Block references [[Note#^id]]	WikilinkMatch.block_id; stored on links_to edge	§5.1.1

Req ID	Requirement (Short)	Implementing Component	Section
FR-022	Transclusion ![[Note]]	prefetchTransclusions() + Preview.tsx expansion	§5.7
FR-030	CommonMark Markdown	unified + remark-parse + remark-gfm	§4.2.1
FR-031	Syntax highlighting	rehype-highlight in preview; CM6 lang packs in editor	§4.1, §4.2
FR-032	Inline LaTeX	remark-math + rehype-katex ($...$)	§4.2.1
FR-033	Display LaTeX	remark-math + rehype-katex ($$...$$)	§4.2.1
FR-034	Split-pane editor/preview	NoteArea with PaneMode; CSS grid layout	§4.2.2
NFR-006	Vault rebuild from .md files	vault_rebuild command → index_full() after DROP	§3.4.1
NFR-009	Keyboard-first operation	CM6 keymap + command palette (Phase 5)	§4.1.1
NFR-010	Light/dark theme	useTheme hook + CSS custom properties	§4.3

8. Phase Exit Criteria

Each phase must satisfy all listed criteria before work on the next phase begins.

8.1 Phase 1 Exit Criteria

1. cargo tauri dev launches without errors.
2. Clicking 'Open Vault' opens a directory picker and initiates indexing.
3. Progress bar in frontend updates during indexing and completes.
4. Note list sidebar shows all .md file titles from the vault.
5. Clicking a note opens its content in a plain textarea (editor stub is acceptable).
6. Typing and saving updates the file on disk (verify with an external text editor).
7. Creating a new note creates a .md file and it appears in the note list.
8. Deleting a note removes it from disk and the list.
9. Modifying a .md file externally updates the note list within 5 seconds.

8.2 Phase 2 Exit Criteria

10. CodeMirror 6 editor renders in the note area with syntax highlighting.
11. Switching between notes does not corrupt editor content.
12. Split-pane mode shows editor and rendered Markdown side-by-side.
13. Inline LaTeX (x^2) renders as mathematical notation in the preview pane.
14. Display LaTeX ($$E=mc^2$$) renders as a centered block equation.
15. Fenced code blocks (```rust) render with syntax highlighting in preview.
16. Light and dark themes are switchable; preference persists across app restarts.

8.3 Phase 3 Exit Criteria

17. [[Wikilinks]] in the editor appear with distinct styling (hover preview is a bonus).
18. The Backlinks panel populates with linked mentions when a note with incoming links is open.
19. Unlinked Mentions section shows plain-text references to the current note.
20. Dangling wikilinks render in a distinct style (orange/dashed).
21. Clicking a dangling link prompts 'Create Note' and creates the note.
22. Renaming a note via note_rename updates all [[wikilinks]] in the vault.
23. refactor_done toast shows the number of files updated.
24. ![[Embedded Note]] renders the content of the target note inline in preview.

9. Open Questions and Deferred Decisions

Question	Deferred To
Should note_save auto-save on a timer (background) or only on explicit Ctrl+S?	Phase 2 implementation — recommend debounced auto-save with manual save override
Should note IDs be SHA-256 of path or UUIDs?	Decided: SHA-256 of vault-relative path (see §2.2). Revisit if cross-vault linking is added.
tantivy for full-text search vs. SurrealDB SEARCH ANALYZER?	Phase 5 — evaluate both; start with SurrealDB SEARCH ANALYZER for simplicity
How should note_rename handle frontmatter title field sync?	Phase 3 implementation — recommended: also update the title field in the frontmatter block
Should wikilink resolution fall back to fuzzy (Levenshtein) matching?	Phase 3 — implement exact match first; add fuzzy fallback if user testing shows friction
Android file watcher: notify polling vs. explicit re-scan on app foreground?	Phase 9 — validate notify polling behaviour on Android; add manual re-scan as fallback
Multi-vault support (multiple vault paths open simultaneously)?	Post-MVP — AppState.vault_path is currently single; multi-vault requires significant refactor

10. Appendix — Dependency Inventory (Phases 1–3)

10.1 Rust Crates (Cargo.toml)

Crate	Version (approx.)	Purpose
tauri	^2.0	Application framework, IPC, plugin system
tauri-plugin-fs	^2.0	Native file read/write
tauri-plugin-dialog	^2.0	Native OS file/folder pickers
tauri-plugin-store	^2.0	Persistent key-value settings storage
surrealdb	^2.0 (kv-surrealkv)	Embedded graph database
tokio	^1, features=[full]	Async runtime
serde	^1, features=[derive]	Serialization for IPC payloads
serde_json	^1	JSON serialization
serde_yaml	^0.9	YAML frontmatter deserialization
pulldown-cmark	^0.11	Markdown parsing (used for H1 title extraction)
regex	^1	Wikilink and frontmatter pattern matching
notify	^6	Cross-platform file system watching
sha2	^0.10	SHA-256 note ID generation
hex	^0.4	Hex encoding for note IDs
chrono	^0.4	Date/time for daily notes and timestamps
thiserror	^1	Ergonomic error type derivation

10.2 Frontend npm Packages (package.json)

Package	Version (approx.)	Purpose
@tauri-apps/api	^2.0	TypeScript IPC bindings (invoke, listen)
@tauri-apps/plugin-dialog	^2.0	Folder picker
@tauri-apps/plugin-store	^2.0	Persistent settings
react	^18	UI component framework
react-dom	^18	React DOM renderer
typescript	^5	Type-safe JavaScript
vite	^5	Development server and production bundler
@codemirror/view	^6	CM6 editor view
@codemirror/state	^6	CM6 state management

Package	Version (approx.)	Purpose
@codemirror/lang-markdown	^6	Markdown language support
@codemirror/language-data	^6	Language pack collection for fenced code
@codemirror/commands	^6	Default key bindings (history, etc.)
@codemirror/theme-one-dark	^6	Dark theme for CM6
unified	^11	Remark/rehype pipeline orchestrator
remark-parse	^11	Markdown parser plugin
remark-gfm	^4	GitHub Flavoured Markdown (tables, tasks)
remark-math	^6	Math token parser
remark-rehype	^11	Remark → rehype bridge
rehype-katex	^7	KaTeX math renderer
rehype-highlight	^7	Syntax highlight for code blocks
rehype-stringify	^10	HTML serializer
katex	^0.16	LaTeX rendering engine
tailwindcss	^3	Utility CSS framework
use-debounce	^10	Debounced auto-save hook

END OF DOCUMENT

CONFIDENTIAL | GraphNotes TDD v1.0 | June 2026